

Bazele Inteligenței Artificiale

Lab 5. Reducerea dimensionalității

Efrem Dragoș-Sebastian-Mihaly

Să zicem că vrem să prezicem riscul de moarte al unei persoane pe baza unor features

[1] Înălțime	[2] Greutate	[3] Tensiunea arterială medie	[4] Pulsul mediu	[5] IMC (Indicele de masă corporală)	[6] Nivelul colesterolului	[7] Media țigărilor consumate / zi	...	[200] Nivelul zahărului	Risc
150	80	140/90	63	36	5	0		99	Slab
174	90	90/60	100	32	4.0	0		95	Moderat
180	82	92/60	78	25	3.7	10		112	Ridicat
...	...	...	...						...

Caracteristici / Attribute / Features

Etichetă / Clasă

Să zicem că vrem să prezicem riscul de moarte al unei persoane pe baza unor features

[1] Înălțime	[2] Greutate	[3] Tensiunea arterială medie	[4] Pulsul mediu	[5] IMC (Indicele de masă corporală)	[6] Nivelul colesterolului	[7] Media țigărilor consumate / zi	...	[200] Nivelul zahărului	Risc
150	80	140/90	63	36	5	0		99	0
174	90	90/60	100	32	4.0	0		95	1
180	82	92/60	78	25	3.7	10		112	2
...	...	...	...						...

Caracteristici / Attribute / Features

Etichetă / Clasă

- 0 - Risc slab
- 1- Risc moderat
- 2 - Risc ridicat

- Ca să înțelegem datele ar trebui să le vizualizăm, dar setul de date precedent conține 200 de features
- Asta ar însemna să vizualizăm lucrurile din 200D, dar deja din 4D ne cam doare capul
- Mai mult, prea multe features duc la „Blestemul dimensionalității”
- Acesta spune că cu cât crește numărul de features (dimensiuni), numărul de date pe care trebuie să le adunăm pentru antrenare pentru o generalizare optimă crește exponențial

Blestemul dimensionalității

Imaginează-ți că ai un punct în spațiu:

- în 1D, e doar un punct pe o linie
- în 2D, punctul e într-un plan (ca pe o foaie de hârtie)
- în 3D, punctul e într-un cub

Dacă mergem mai departe în 4D și peste, spațiul disponibil pentru date crește atât de mult încât punctele din setul de date devin din ce în ce mai departe unele de altele.

Astfel, algoritmi (cum sunt cei de clasificare sau de regresie) au dificultăți în a găsi structuri sau corelații între aceste puncte.

Blestemul dimensionalității

„Frica punctelor de a fi izolate în spațiu”

- Cassie Kozyrkhov, Google

Blestemul dimensionalității

- În dimensiuni mari, majoritatea caracteristicilor sunt irelevante sau zgomot.
- Asta face ca algoritmi să „învețe” modele care nu generalizează bine pe date noi, ci doar se potrivesc pe datele de antrenament, adică care fac overfitting.
- Mai mult, e dificil să avem un număr suficient de date pentru a acoperi un spațiu cu multe dimensiuni.
- Așa că algoritmi ajung să învețe pe fragmente mici, nefiind capabili să construiască o înțelegere completă a datelor.
- Pe măsură ce cresc dimensiunile, crește și complexitatea de calcul. Algoritmi care ar funcționa rapid în 2D sau 3D devin foarte lenti în 10D sau 100D, uneori imposibil de utilizat la scara asta.

Blestemul dimensionalității

- Un fact interesant al geometriei multidimensionale:
 - În spații N-dimensionale, cu N destul de mare, distanțele dintre oricare două puncte tind să devină mai uniforme
 - Deci, cu cât creștem numărul de dimensiuni, cu atât distanțele dintre oricare două puncte tind să se apropie de o valoare unică
- Consecință:
 - Această concentrare a distanțelor pune dificultăți la modelele bazate pe distanțe, cum ar fi SVM sau K-Nearest Neighbours

[1] Înălțime	[2] Greutate	[3] Tensiunea arterială medie	[4] Pulsul mediu	[5] IMC (Indicele de masă corporală)	[6] Nivelul colesterolului	[7] Media țigărilor consumate / zi	...	[200] Nivelul zahărului	Risc
150	80	140/90	63	36	5	0		99	0
174	90	90/60	100	32	4.0	0		95	1
180	82	92/60	78	25	3.7	10		112	2
...	...	...	...						...

Caracteristici / Atribute / Features

Etichetă / Clasă

Care e soluția?

- Reducerea numărului de features
- Să păstrăm cât mai multă informație utilă posibilă

Principal Component Analysis (PCA)

Principal Component Analysis

- Scopul este de a reduce numărul de features, dar să păstrăm cât mai multă informație.
- Algoritmul:
 - Primește ca intrare N features
 - Creează tot N features, numite componente principale
 - Componentele principale sunt create astfel încât dacă iei doar primele 1,2,3 (puține) componente, acestea să conțină cât mai multă informație

[1] Înălțime	[2] Greutate	[3] Tensiunea arterială medie	[4] Pulsul mediu	[5] IMC (Indicele de masă corporală)	[6] Nivelul colesterolului	[7] Media țigărilor consumate / zi	...	[200] Nivelul zahărului	Risc
150	80	140/90	63	36	5	0		99	0
174	90	90/60	100	32	4.0	0		95	1
180	82	92/60	78	25	3.7	10		112	2
...	...	...	...						...

$$\vec{x}_1 = \begin{bmatrix} 150 \\ 80 \\ 140/90 \\ 63 \\ 36 \\ 5.0 \\ 0 \\ \dots \\ 99 \end{bmatrix}, \quad \vec{x}_2 = \begin{bmatrix} 174 \\ 90 \\ 90/60 \\ 100 \\ 32 \\ 4.0 \\ 0 \\ \dots \\ 95 \end{bmatrix}, \quad \vec{x}_3 = \begin{bmatrix} 180 \\ 82 \\ 92/60 \\ 78 \\ 25 \\ 3.7 \\ 10 \\ \dots \\ 112 \end{bmatrix} \quad \dots$$

Prima componentă principală:

$$\vec{u} = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_{200} \end{bmatrix}$$

$$\vec{x}_1 = \begin{bmatrix} 150 \\ 80 \\ 140/90 \\ 63 \\ 36 \\ 5.0 \\ 0 \\ \dots \\ 99 \end{bmatrix}, \quad \vec{x}_2 = \begin{bmatrix} 174 \\ 90 \\ 90/60 \\ 100 \\ 32 \\ 4.0 \\ 0 \\ \dots \\ 95 \end{bmatrix}, \quad \vec{x}_3 = \begin{bmatrix} 180 \\ 82 \\ 92/60 \\ 78 \\ 25 \\ 3.7 \\ 10 \\ \dots \\ 112 \end{bmatrix} \quad \dots$$

Prima componentă principală:

$$\vec{u} = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_{200} \end{bmatrix}$$

- Putem considera fiecare exemplar din setul nostru de date N-dimensional ca un vector într-un spațiu N-dimensional
- Prima componentă principală este tot un vector în acel spațiu
- Acest vector nou are proprietatea că, dacă proiectăm fiecare vector din setul nostru de date pe el, magnitudinea vectorilor proiectați este maximizată pentru toți vectorii

Prima componentă principală:

$$\vec{x}_1 = \begin{bmatrix} 150 \\ 80 \\ 140/90 \\ 63 \\ 36 \\ 5.0 \\ 0 \\ \dots \\ 99 \end{bmatrix}, \quad \vec{x}_2 = \begin{bmatrix} 174 \\ 90 \\ 90/60 \\ 100 \\ 32 \\ 4.0 \\ 0 \\ \dots \\ 95 \end{bmatrix}, \quad \vec{x}_3 = \begin{bmatrix} 180 \\ 82 \\ 92/60 \\ 78 \\ 25 \\ 3.7 \\ 10 \\ \dots \\ 112 \end{bmatrix} \quad \dots \quad \vec{u} = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_{200} \end{bmatrix}$$

- Prin mărimea vectorilor proiectați măsurăm informația rămasă din acei vectori după proiecție
- Ne interesează ca atunci când avem toți vectorii proiectați pe prima componentă, mărimea proiecțiilor să fie cât mai împrăștiate
- Împrăștierea aceasta mai poartă numele și de variație. Ne dorim o variație cât mai mare deoarece astfel păstrăm cât mai multă informație din toate punctele

$$\vec{x}_1 = \begin{bmatrix} 150 \\ 80 \\ 140/90 \\ 63 \\ 36 \\ 5.0 \\ 0 \\ \dots \\ 99 \end{bmatrix}, \quad \vec{x}_2 = \begin{bmatrix} 174 \\ 90 \\ 90/60 \\ 100 \\ 32 \\ 4.0 \\ 0 \\ \dots \\ 95 \end{bmatrix}, \quad \vec{x}_3 = \begin{bmatrix} 180 \\ 82 \\ 92/60 \\ 78 \\ 25 \\ 3.7 \\ 10 \\ \dots \\ 112 \end{bmatrix} \quad \dots$$

Prima componentă principală:

$$\vec{u} = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \\ u_{200} \end{bmatrix}$$

- După ce obținem prima componentă principală, a doua componentă principală are proprietatea că este ortogonală pe prima. A treia componentă principală va fi ortogonală pe primele două și așa mai departe
- Dacă nu ar fi ortogonale, atunci două componente s-ar putea să prezinte informații comune, lucru care este evitat
- Se repetă același proces discutat, dar pentru noile componente principale

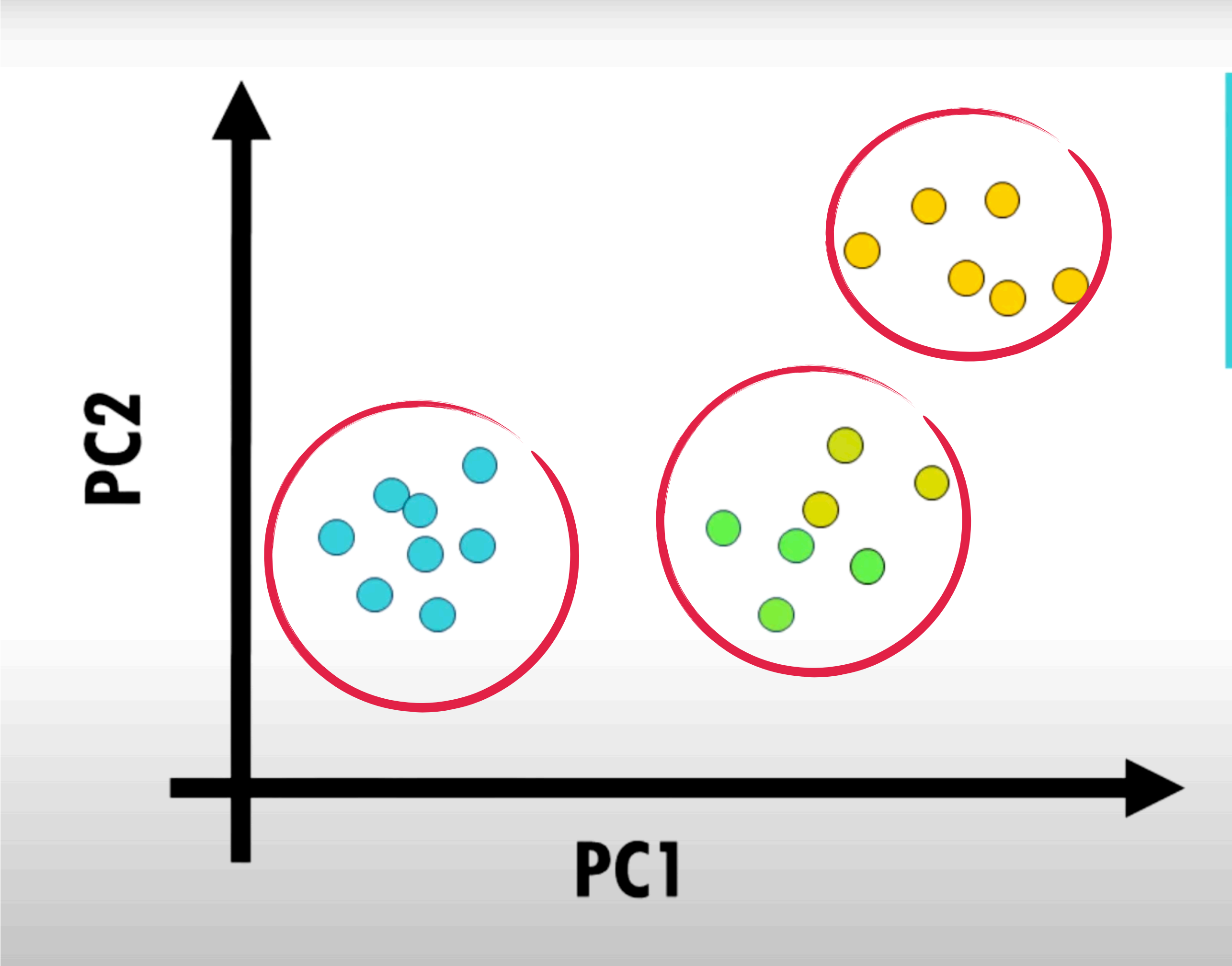
[1] Înălțime	[2] Greutate	[3] Tensiunea arterială medie	[4] Pulsul mediu	[5] IMC (Indicele de masă corporală)	[6] Nivelul colesterolului	[7] Media țigărilor consumate / zi	...	[200] Nivelul zahărului	Risc
150	80	140/90	63	36	5	0		99	0
174	90	90/60	100	32	4.0	0		95	1
180	82	92/60	78	25	3.7	10		112	2
...	...	...	...						...

Ca să ajungem de la 200D la 2D de exemplu, vom păstra doar primele două componente principale

Așa va arăta setul de date acum:

[1] PC1	[2] PC2	Risc
-1	3	0
2	4	1
3	2	2
...	...	...

Aşa va arăta setul de date acum:



[1] PC1	[2] PC2	Risc
-1	3	0
2	4	1
3	2	2
...	...	...

- Acum putem observa ușor cum arată datele
- Ne putem da seama că pentru clasificare putem aplica un algoritm gen K-Nearest Neighbours sau SVM
- Grupurile observate nu erau vizibile în spații mai mari

- Grupurile observate nu erau vizibile în spații mai mari

- Țineți minte că cu cât creștem numărul de features, cu atât riscăm să izolăm puncte în noul spațiu
- Astfel că atunci când scădem numărul de features, ajungem într-un spațiu mai mic unde punctele sunt mai împreunate
- Astfel ne poate fi mai ușor să găsim tipare sau reguli în date

Principal Component Analysis

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Incarcare set de date Iris
data = load_iris()
X, y = data.data, data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Antrenare fara PCA
clf_no_pca = LogisticRegression(max_iter=200, random_state=42)
clf_no_pca.fit(X_train, y_train)
y_pred_no_pca = clf_no_pca.predict(X_test)
accuracy_no_pca = accuracy_score(y_test, y_pred_no_pca)

# Antrenare cu PCA
pca = PCA(n_components=1)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

clf_pca = LogisticRegression(max_iter=200, random_state=42)
clf_pca.fit(X_train_pca, y_train)
y_pred_pca = clf_pca.predict(X_test_pca)
accuracy_pca = accuracy_score(y_test, y_pred_pca)

# Putem aplica PCA pe un exemplar anume
X_15_reduced = pca.transform([X_test[15]])

# Putem reconstrui acelasi exemplar dupa PCA
X_15_recovered = pca.inverse_transform(X_15_reduced)

accuracy_no_pca, accuracy_pca, pca.n_components_, pca.explained_variance_ratio_, X_test[15], X_15_recovered
```

Încărcare set Iris

Split 75% train, 25% test

Antrenare și testare fără PCA

Antrenare și testare cu PCA

```
(1.0,
0.9736842105263158,
1,
array([0.91835496]),
array([6.3, 3.3, 4.7, 1.6]),
array([[6.20987566, 2.95751625, 4.71063993, 1.5901021 ]]))
```

Principal Component Analysis

```
pca = PCA(n_components=1)
```



PCA îmi va da doar prima componentă
Astfel reducem dimensionalitatea la 1D

- Dacă foloseam `n_components=2` mi-ar fi dat primele două și tot așa
- Putem da și valori < 1 ca parametru, de exemplu `n_components=0.95`
 - În acest caz, PCA îmi va da atâtea componente necesare astfel încât să se păstreze 95% informație din setul original, dacă se poate

Principal Component Analysis

```
pca = PCA(n_components=1)
```

- Pe acest obiect, având un set de date X , putem aplica diverse metode:
- `pca.fit(X)` îmi calculează componentele principale pe baza lui X
- `X_new = pca.transform(X)` folosește componentele principale calculate anterior ca să transforme setul de date X astfel încât să conțină dimensionalitatea cerută
- Cele două pot fi combinate într-o singură linie: `X_new = pca.fit_transform(X)`

Principal Component Analysis

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Incarcare set de date Iris
data = load_iris()
X, y = data.data, data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Antrenare fara PCA
clf_no_pca = LogisticRegression(max_iter=200, random_state=42)
clf_no_pca.fit(X_train, y_train)
y_pred_no_pca = clf_no_pca.predict(X_test)
accuracy_no_pca = accuracy_score(y_test, y_pred_no_pca)

# Antrenare cu PCA
pca = PCA(n_components=1)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

clf_pca = LogisticRegression(max_iter=200, random_state=42)
clf_pca.fit(X_train_pca, y_train)
y_pred_pca = clf_pca.predict(X_test_pca)
accuracy_pca = accuracy_score(y_test, y_pred_pca)

# Putem aplica PCA pe un exemplar anume
X_15_reduced = pca.transform([X_test[15]])

# Putem reconstrui acelasi exemplar dupa PCA
X_15_recovered = pca.inverse_transform(X_15_reduced)

accuracy_no_pca, accuracy_pca, pca.n_components, pca.explained_variance_ratio, X_test[15], X_15_recovered
```

Aplicăm fit_transform pe train
Trebuie transformat și setul de test

```
(1.0,
0.9736842105263158,
1,
array([0.91835496]),
array([6.3, 3.3, 4.7, 1.6]),
array([[6.20987566, 2.95751625, 4.71063993, 1.5901021 ]]))
```

Principal Component Analysis

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Incarcare set de date Iris
data = load_iris()
X, y = data.data, data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Antrenare fara PCA
clf_no_pca = LogisticRegression(max_iter=200, random_state=42)
clf_no_pca.fit(X_train, y_train)
y_pred_no_pca = clf_no_pca.predict(X_test)
accuracy_no_pca = accuracy_score(y_test, y_pred_no_pca)

# Antrenare cu PCA
pca = PCA(n_components=1)
X_train_pca = pca.fit_transform(X_train)

X_test_pca = pca.transform(X_test)

clf_pca = LogisticRegression(max_iter=200, random_state=42)
clf_pca.fit(X_train_pca, y_train)
y_pred_pca = clf_pca.predict(X_test_pca)
accuracy_pca = accuracy_score(y_test, y_pred_pca)

# Putem aplica PCA pe un exemplar anume
X_15_reduced = pca.transform([X_test[15]])

# Putem reconstrui acelasi exemplar dupa PCA
X_15_recovered = pca.inverse_transform(X_15_reduced)

accuracy_no_pca, accuracy_pca, pca.n_components_, pca.explained_variance_ratio_, X_test[15], X_15_recovered
```

```
(1.0,
0.9736842105263158,
1,
array([0.91835496]),
array([6.3, 3.3, 4.7, 1.6]),
array([[6.20987566, 2.95751625, 4.71063993, 1.5901021 ]]))
```

- Prima dată facem fit_transform pe setul de train
- Asta face ca PCA să obțină componentele principale doar pe baza datelor din train, și după convertește setul de train astfel încât să aibă n_components features
- Dar setul de test rămâne cel vechi, așa că trebuie să îl transformăm și pe el astfel încât să conțină n_components features

Principal Component Analysis

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Incarcare set de date Iris
data = load_iris()
X, y = data.data, data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Antrenare fara PCA
clf_no_pca = LogisticRegression(max_iter=200, random_state=42)
clf_no_pca.fit(X_train, y_train)
y_pred_no_pca = clf_no_pca.predict(X_test)
accuracy_no_pca = accuracy_score(y_test, y_pred_no_pca)

# Antrenare cu PCA
pca = PCA(n_components=1)
X_train_pca = pca.fit_transform(X_train)

X_test_pca = pca.transform(X_test)

clf_pca = LogisticRegression(max_iter=200, random_state=42)
clf_pca.fit(X_train_pca, y_train)
y_pred_pca = clf_pca.predict(X_test_pca)
accuracy_pca = accuracy_score(y_test, y_pred_pca)

# Putem aplica PCA pe un exemplar anume
X_15_reduced = pca.transform([X_test[15]])

# Putem reconstrui acelasi exemplar dupa PCA
X_15_recovered = pca.inverse_transform(X_15_reduced)

accuracy_no_pca, accuracy_pca, pca.n_components_, pca.explained_variance_ratio_, X_test[15], X_15_recovered
```

Transformăm doar al 16-lea exemplar din setul de test

```
(1.0,
 0.9736842105263158,
 1,
 array([0.91835496]),
 array([6.3, 3.3, 4.7, 1.6]),
 array([[6.20987566, 2.95751625, 4.71063993, 1.5901021 ]]))
```

- Putem apela `inverse_transform` pe ceva redus ca să îl recuperăm
- Adică să trecem de la dimensionalitatea mică dată de PCA la cea originală
- Trecerea de la dimensiunea mare la dimensiunea mică duce deja la pierdere de informație, așa că atunci când recuperăm vectorul s-ar putea să nu fie același cu cel așteptat

Principal Component Analysis

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Incarcare set de date Iris
data = load_iris()
X, y = data.data, data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Antrenare fara PCA
clf_no_pca = LogisticRegression(max_iter=200, random_state=42)
clf_no_pca.fit(X_train, y_train)
y_pred_no_pca = clf_no_pca.predict(X_test)
accuracy_no_pca = accuracy_score(y_test, y_pred_no_pca)

# Antrenare cu PCA
pca = PCA(n_components=1)
X_train_pca = pca.fit_transform(X_train)

X_test_pca = pca.transform(X_test)

clf_pca = LogisticRegression(max_iter=200, random_state=42)
clf_pca.fit(X_train_pca, y_train)
y_pred_pca = clf_pca.predict(X_test_pca)
accuracy_pca = accuracy_score(y_test, y_pred_pca)

# Putem aplica PCA pe un exemplar anume
X_15_reduced = pca.transform([X_test[15]])

# Putem reconstrui acelasi exemplar dupa PCA
X_15_recovered = pca.inverse_transform(X_15_reduced)

accuracy_no_pca, accuracy_pca, pca.n_components_, pca.explained_variance_ratio_, X_test[15], X_15_recovered
```

(1.0, 0.9736842105263158, 1, array([0.91835496]), array([6.3, 3.3, 4.7, 1.6]), array([[6.20987566, 2.95751625, 4.71063993, 1.5901021]]))

- Am afișat:
 - Acuratețea pe test fără PCA
 - Acuratețea pe test cu PCA
 - Numărul de componente
 - Câtă informație păstrează fiecare componentă
 - Al șaisprezecelea exemplar din setul de test (înainte de PCA)
 - Al șaisprezecelea exemplar din setul de test (trecut prin PCA, și după transpus iară în dimensionalitatea mare)

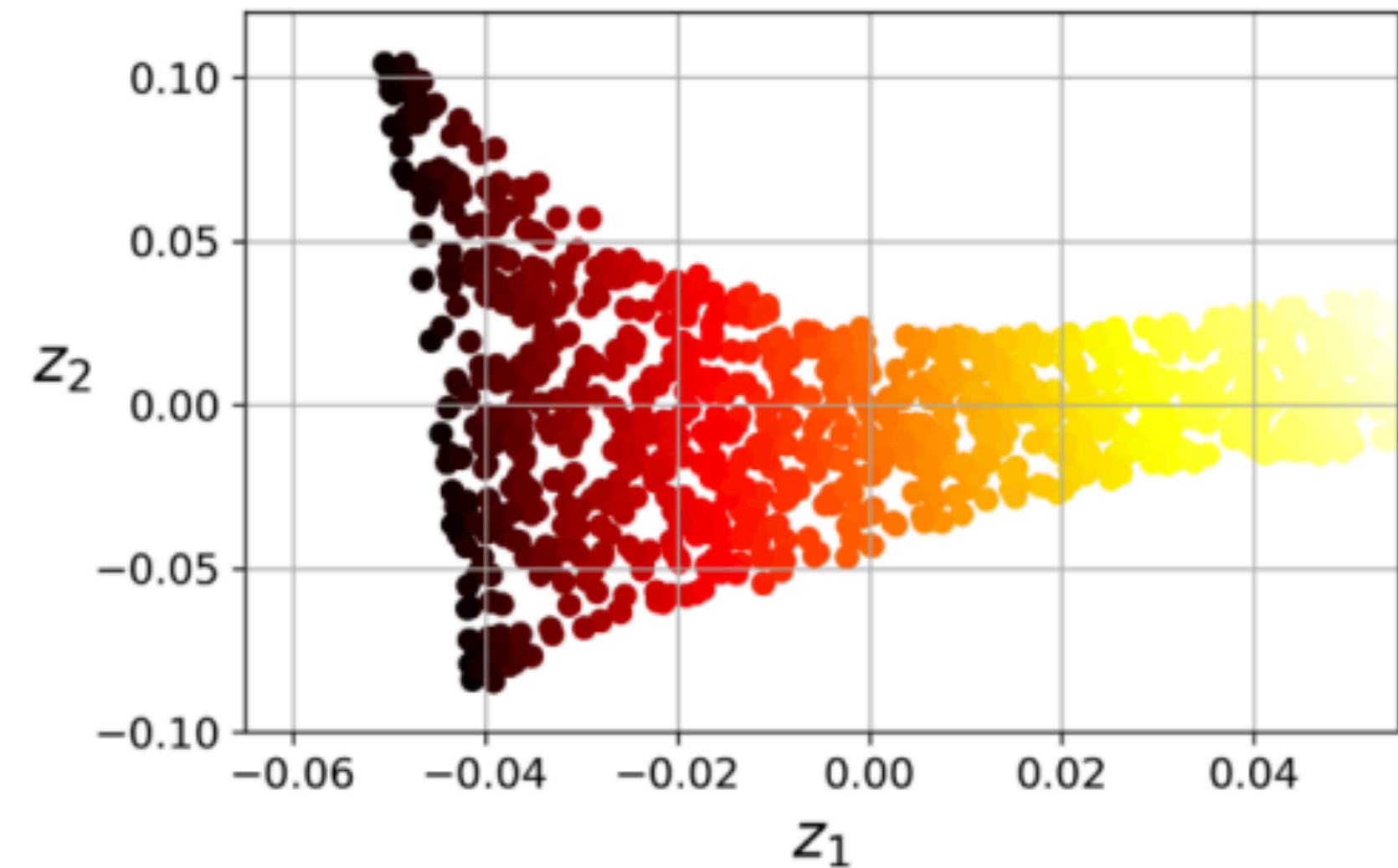
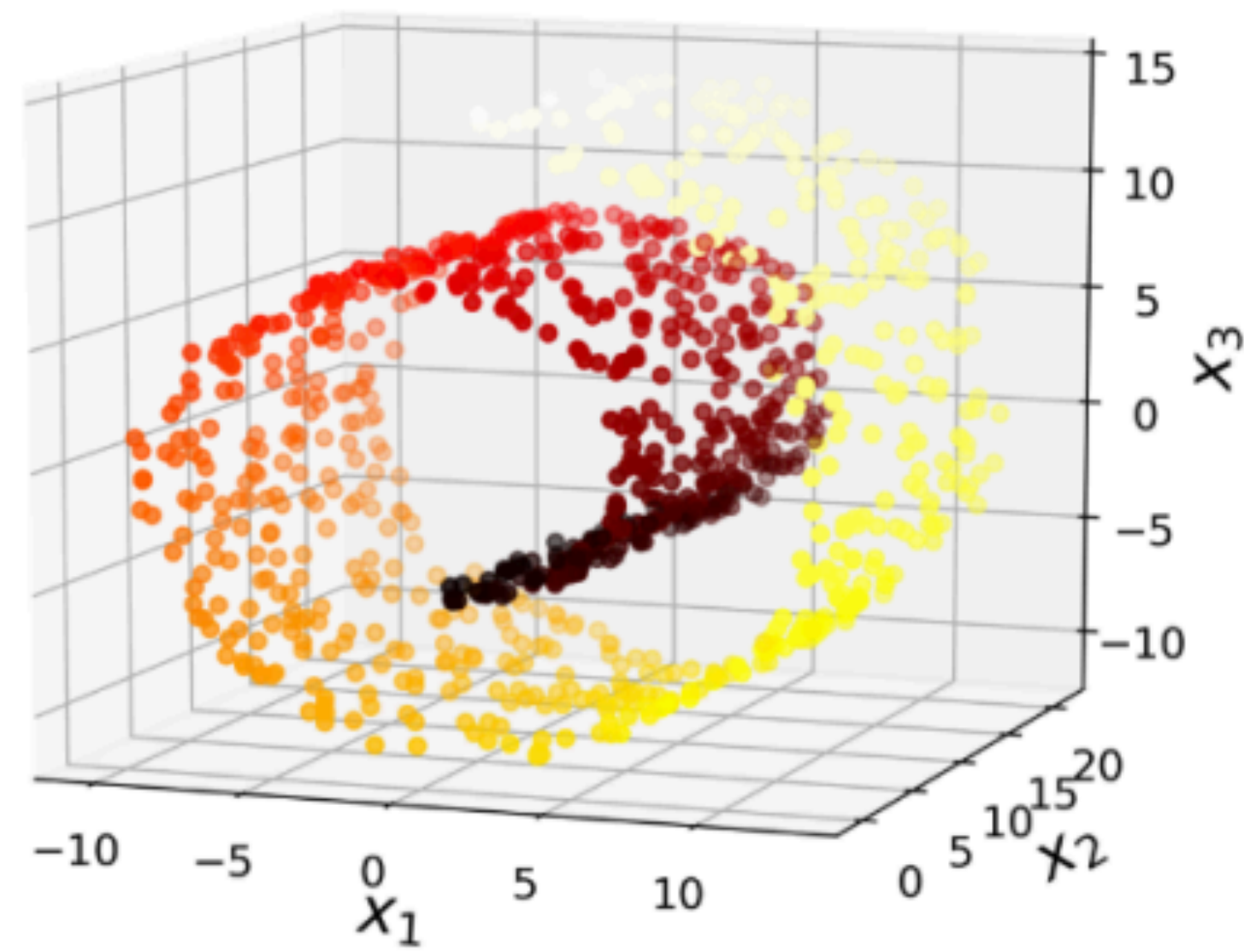
- PCA se uită mereu la ansamblul datelor atunci când obține o componentă principală
- Altă metodă, numită LLE (Locally Linear Embedding), se uită la relațiile locale dintre puncte
- LLE, la fel ca PCA, contribuie la reducerea dimensionalității, dar pe alt principiu
- LLE identifică *vecinii* fiecărui punct și încearcă să păstreze această structură locală în noua dimensiune mai mică.

- Pentru fiecare punct x_i din date, LLE determină K vecini apropiați (unde K e un număr prestabilit). Vecinii sunt identificați de obicei pe baza distanței euclidiene.
- Apoi, LLE încearcă să reconstruiască fiecare punct x_i ca o combinație liniară a vecinilor săi. Practic, găsește coeficienți de ponderare w_{ij} astfel încât:

$$x_i \approx \sum_{j \in \mathbf{Vecinii}(i)} w_{ij} x_j$$

- După ce sunt obținute ponderile w_{ij} în spațiul original, se caută un set de puncte în noul spațiu (cu dimensiune redusă) care păstrează aceste relații de reconstrucție. Scopul este să obținem punctele y_i într-un spațiu de dimensiune redusă astfel încât:

$$y_i \approx \sum_{j \in \mathbf{Vecinii}(i)} w_{ij} y_j$$



- Util când apare efectul de manifold
- Un manifold este ca o suprafață de dimensiune mai mică care se întinde într-un spațiu de dimensiuni mai mari
- De exemplu, poza din stânga poate fi văzută ca o foaie de hârtie (un plan în 2D) dar curbată astfel încât să arate în 3D

Locally Linear Embedding

```
✓ 0 sec. ▶ from sklearn.datasets import make_swiss_roll
from sklearn.model_selection import train_test_split
from sklearn.manifold import LocallyLinearEmbedding

X, _ = make_swiss_roll(n_samples=1000, noise=0.1)

X_train, X_test = train_test_split(X, test_size=0.25, random_state=42)

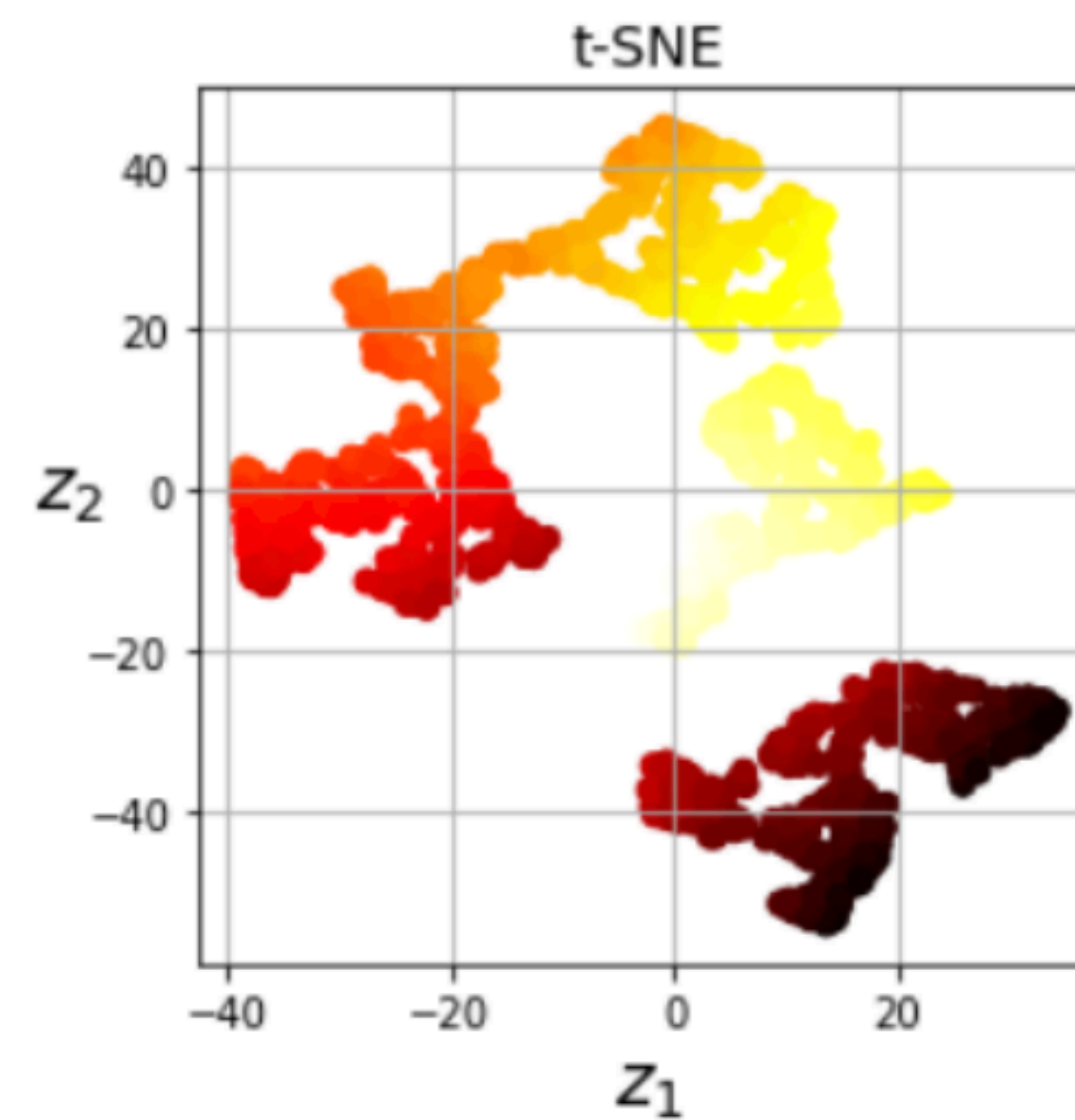
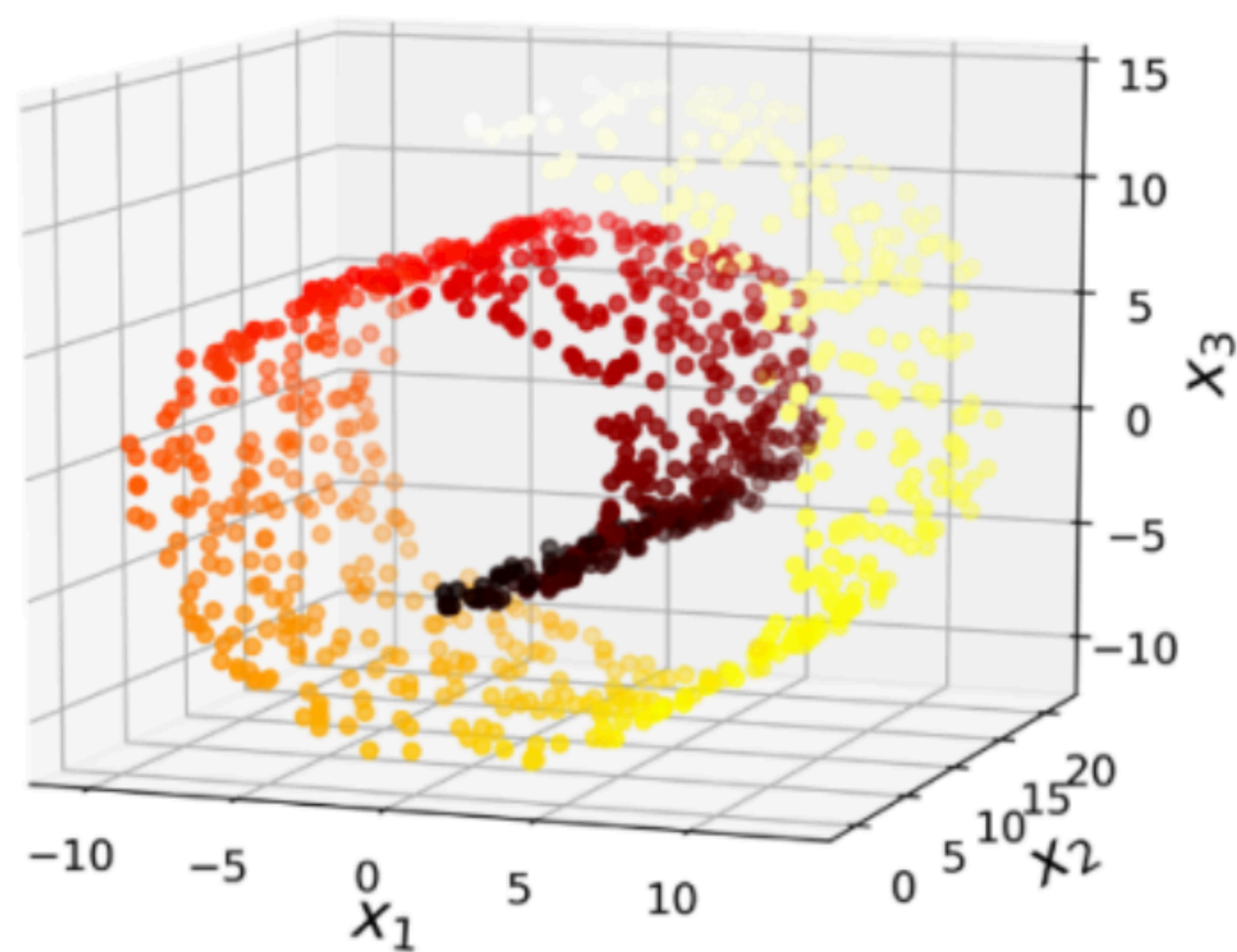
lle = LocallyLinearEmbedding(n_neighbors=10, n_components=2, random_state=42)
X_train_lle = lle.fit_transform(X_train)
X_test_lle = lle.transform(X_test)
```

Parametri utili la LocallyLinearEmbedding:

- n_neighbors: câți vecini să ia în calcul
- n_components: dimensionalitatea finală

- Pe lângă PCA și LLE, mai există și t-Distributed stochastic neighbour embedding (t-SNE)
- Acesta este folosit cel mai mult la vizualizarea datelor
- t-SNE reduce dimensionalitatea încercând să mențină exemplarele similare apropiate în noul spațiu și cele disimilare să fie depărtate

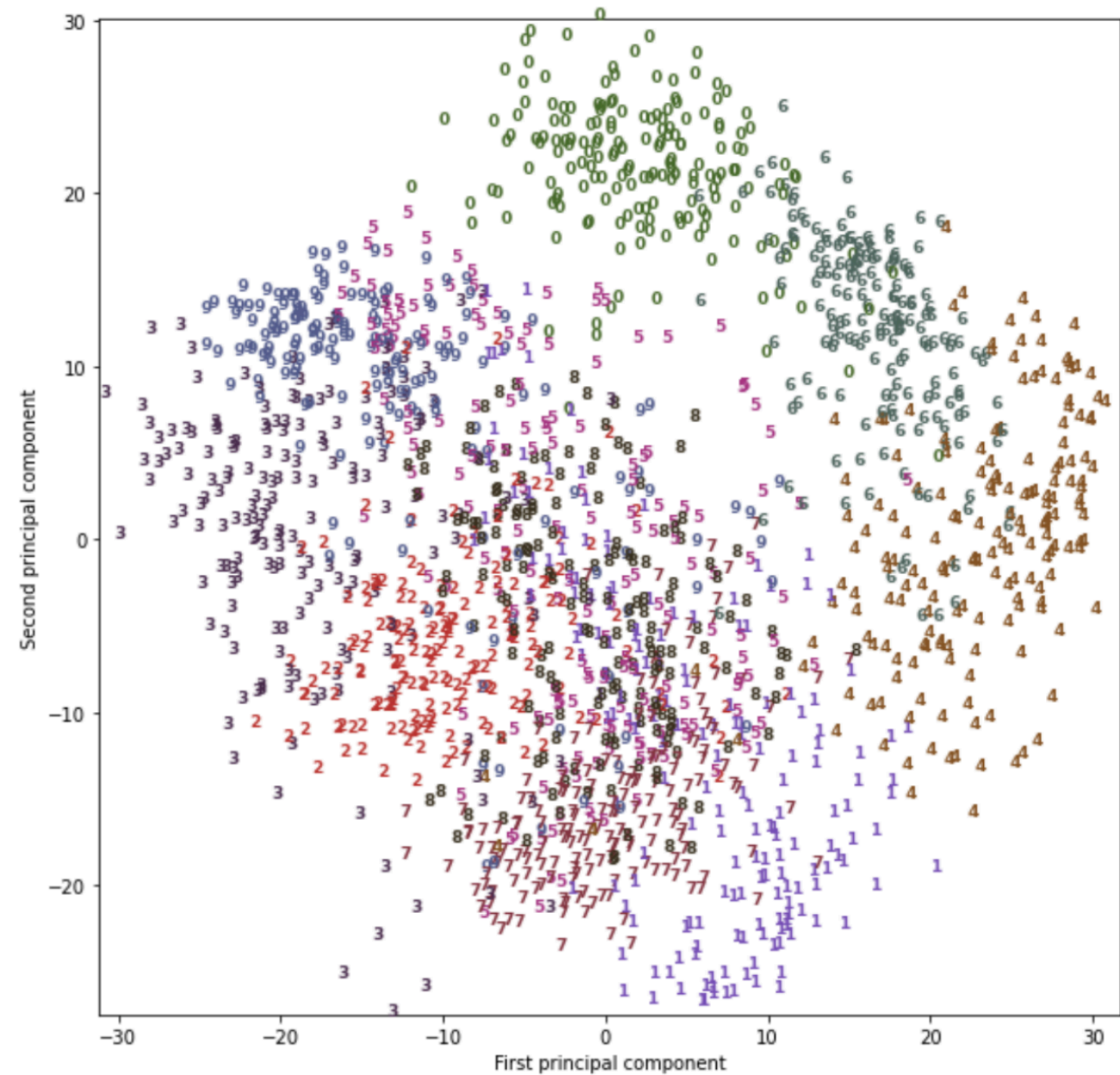
t-SNE



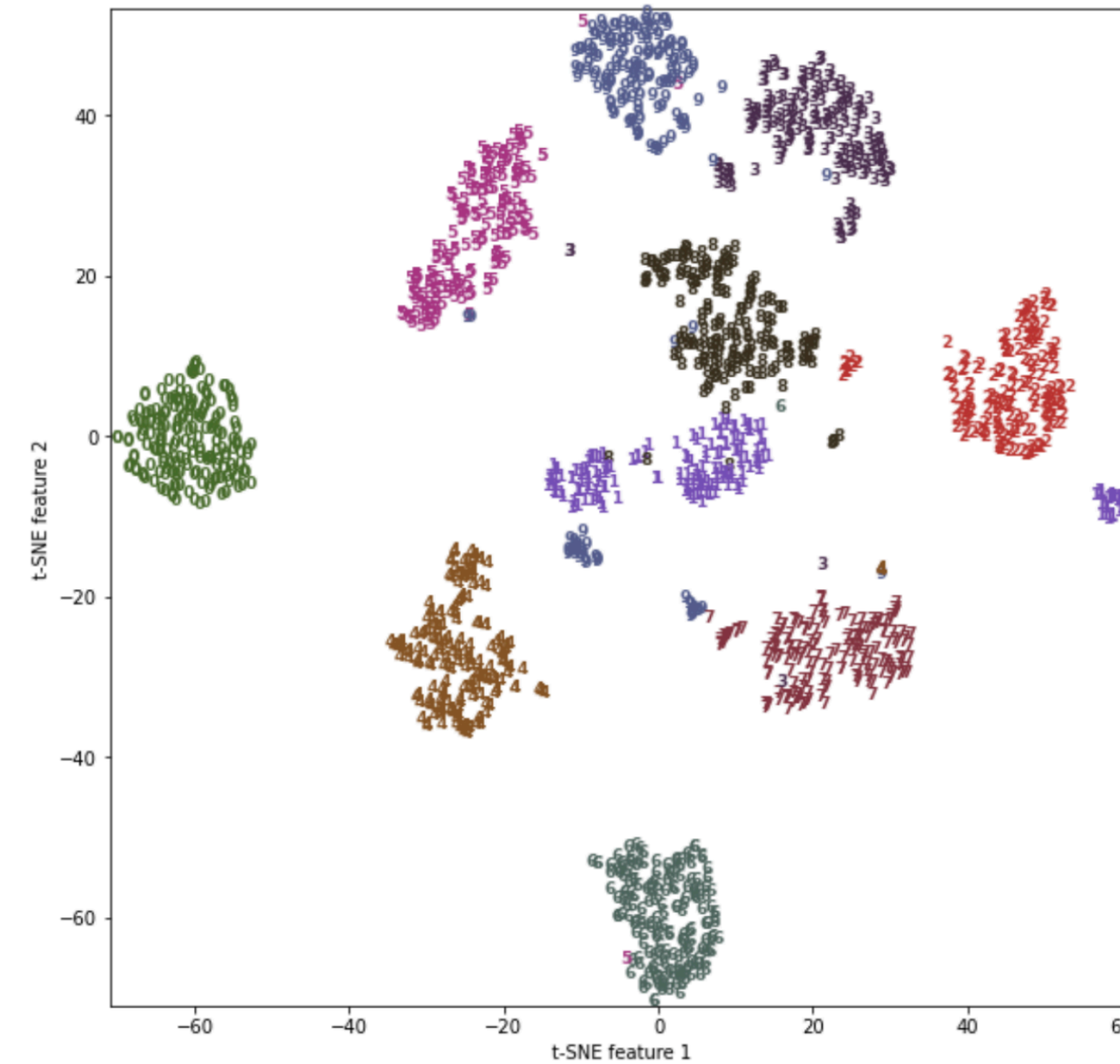
t-SNE

Aplicare pe setul de date MNIST:

- conține poze cu cifre și este folosit pentru a antrena modele simple să detecteze cifre



cu PCA



cu t-SNE

That's it for today!
Now let's do some exercises.

That's it for today!
Now let's do some exercises.

Laboratory 5 – Exercises

1. Use the `iris` dataset in order to reduce its dimensionality down to three dimensions by using the PCA algorithm. Print the principal components and the explained variance ratio and plot the 3D dataset. Also visualize the data reduced to three dimensions using the t-SNE method.
2. Use the `digits` dataset to perform a classification task using Linear Support Vector Machine (SVM). Set the parameter `C` to 5. Split the data such that the test split is 25% of the initial dataset. Compare the image classification task using the original number of features (64) to the reduced features (20) using the PCA algorithm. Compare:
 - the training time for both models;
 - the difference in accuracy score.